The End of Human-Scale AppSec

Rebuilding Security for the AI-Native SDLC

Ken Johnson · CTO & Co-Founder, DryRun Security



DRYRUN SECURITY



ABOUT KEN

I've spent my career building, breaking, and rethinking AppSec.

Ken Johnson · CTO & Co-Founder, DryRun Security

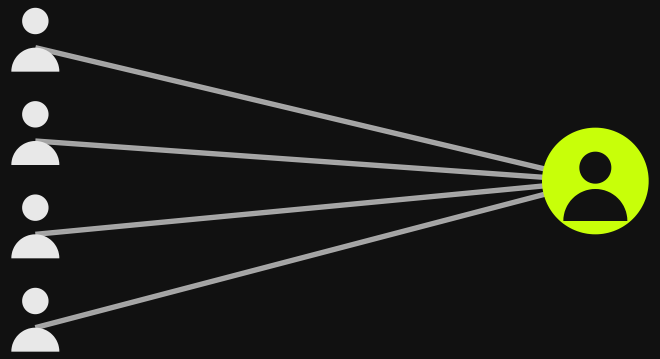
- **19 years** in application security
- Co-host of the Absolute AppSec podcast
- Longtime OWASP contributor; Black Hat and DEF CON trainer
- Led AppSec at GitHub and LivingSocial
- Creator of RailsGoat, WeirdAAL, and WXF
- Now building agentic systems for contextual code security analysis

THE QUESTION

What happens when software stops being produced at human scale?

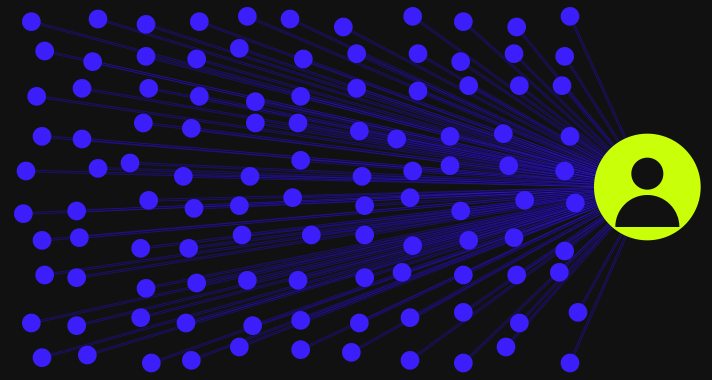
The volume grows. The security program doesn't.

Human scale



A few developers, one security engineer: manageable

Agentic scale



A swarm of agents, the same one security engineer

ACT I

The hidden human architecture of AppSec

Our tools shaped the work

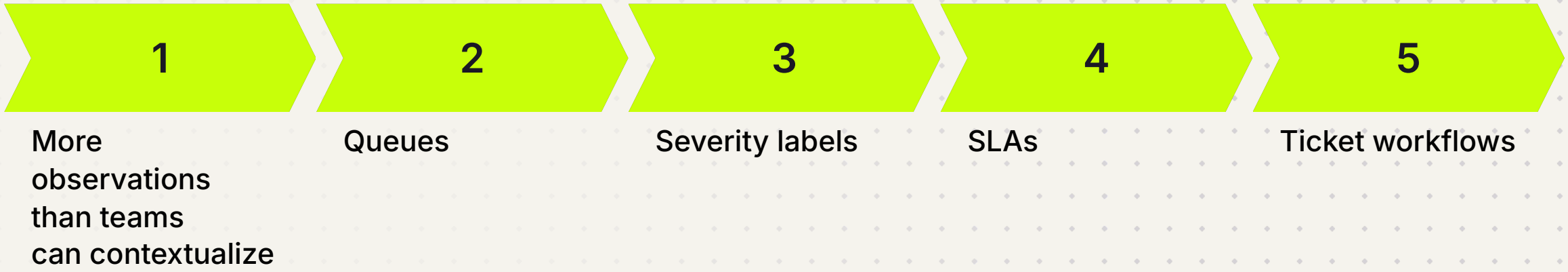
1. Scanner produces observations
2. Human validates
3. Human reconstructs context
4. Human determines impact
5. Human prioritizes
6. Human creates ticket
7. Human negotiates remediation

One step is automated. Six belong to a person.

The practitioner connects everything the tools cannot



The backlog was an adaptation to tool limitations



A rational response, not practitioner failure.

We started managing findings instead of managing risk

WHAT WE MANAGE

A finding backlog

Static rows, counted and aged.

WHAT ACTUALLY MATTERS

A service whose risk keeps changing

New code, new exposure, new dependencies every week.

Findings are evidence about risk, not the risk state.

Finding more vulnerabilities does not prove that risk went down

ACTIVITY

- Findings created
- Tickets opened
- Scans completed
- Mean time to triage

OUTCOMES

- Faster Remediation
- Controls verified
- Exposure reduced
- Unsafe patterns prevented
- High-risk services improved

AppSec exists to reduce risk in software

BUILD IT SAFE

Secure design, threat modeling, guardrails and paved roads, secure frameworks and libraries

CHECK IT

Code and architecture review, security testing, developer guidance

RESPOND AND GOVERN

External report validation, incident support, organizational risk management

ACT II

The human integration layer no longer scales

Two things are changing at the same time

01

AI is changing how software is delivered

More code, more services, more changes, produced faster than people can read them.

+

02

AI can interpret and synthesize data in novel ways

Systems can now do security work our old tools never could.

More code is only half of the story.

The development side is ambitious

AI-native development visions place agents across the entire lifecycle.

01 Planning

02 Design

03 Implementation

04 Testing

05 Review

06 Deployment

07 Maintenance

Every stage gets agents.

Security gets documentation and another review step

01 Update security requirements

02 Update a threat model

03 Run a review tool

04 Produce findings

A broad agentic system narrows into four small steps.

Example: Anthropic published AI-native SDLC playbook.
https://claude.com/resources/articles/the-ai-native-sdlc-playbook

We are not bug janitors

- Design
- Architecture
- Guardrails
- Education

AppSec

`scanning` `review`

`findings` `remediation`

- Incident response
- Consultation
- External reports
- Organizational risk

We are rebuilding yesterday's tools with more AI

SAST

+ AI

DAST

+ AI

SCA

+ AI

Tickets

+ AI

Dashboards

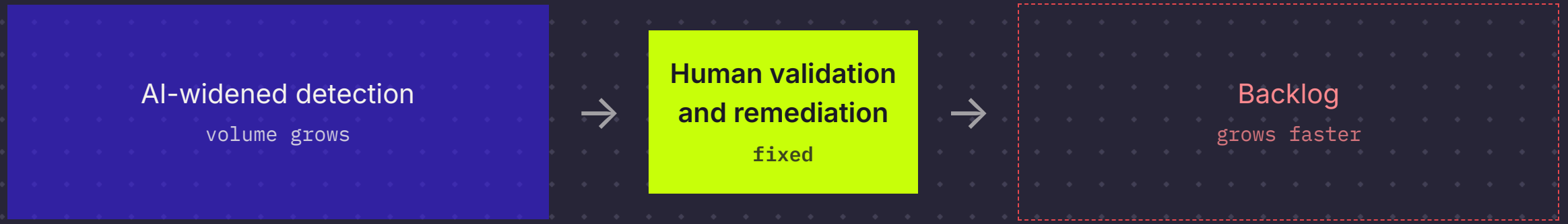
+ AI

Review gates

+ AI

Agents added. The underlying work unchanged.

Faster discovery can create a faster backlog



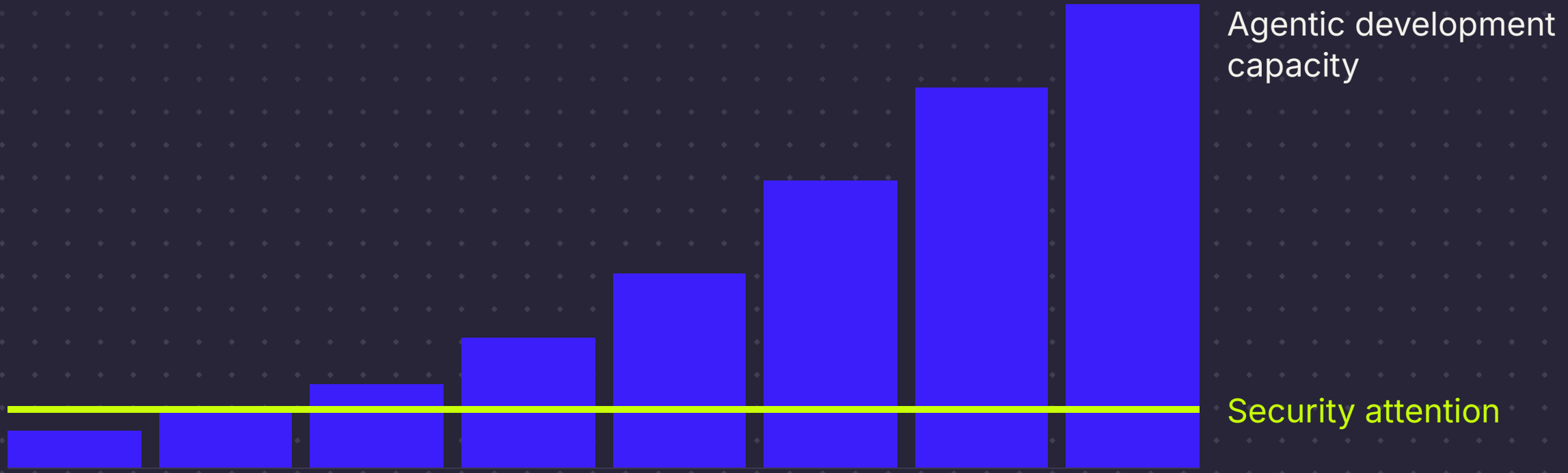
Why "AI-powered D/I/S-AST" can amplify the old failure.

Our processes assume humans are the primary producers

- Humans write most changes, at reviewable velocity
- Security can sample the important work
- A human can reconstruct missing context
- Current processes remain scalable controls

Four assumptions hiding in every AppSec program.

Software production expands. The human control plane does not.



You cannot hire your way through a broken operating model

Doing the work doesn't scale. Governing the work can.

Humans doing the work

- Experts personally run every threat model, design consult, and risk decision
- Capacity grows one hire at a time
- Agentic change outpaces it

Humans governing agentic work

Practitioner governs

- Define security properties
- Build guardrails and paved roads
- Set evidence and escalation rules
- Evaluate the system

Agent workflows

- Gather context
- Maintain knowledge state
- Verify controls
- Track service risk state

Capacity grows with the agents

We can produce a larger backlog, or change the operating model

AI-scale software production

PATH 1 · MORE OF THE SAME

More generation

More scanning

More findings

More human triage

Larger backlog

PATH 2 · CHANGE THE MODEL

Continuous context

Reasoned investigation

Verified controls

↻ Changing application risk state

ACT III

Change the center of the system: continuous risk-state management

From vulnerability backlog to application risk state

OLD CENTER

Finding backlog

→

NEW CENTER

Evolving risk state of the service

Risk is the totality of what we know about the service

SHAPE

- Architecture and trust boundaries
- Exposure and data sensitivity
- Ownership and operational health

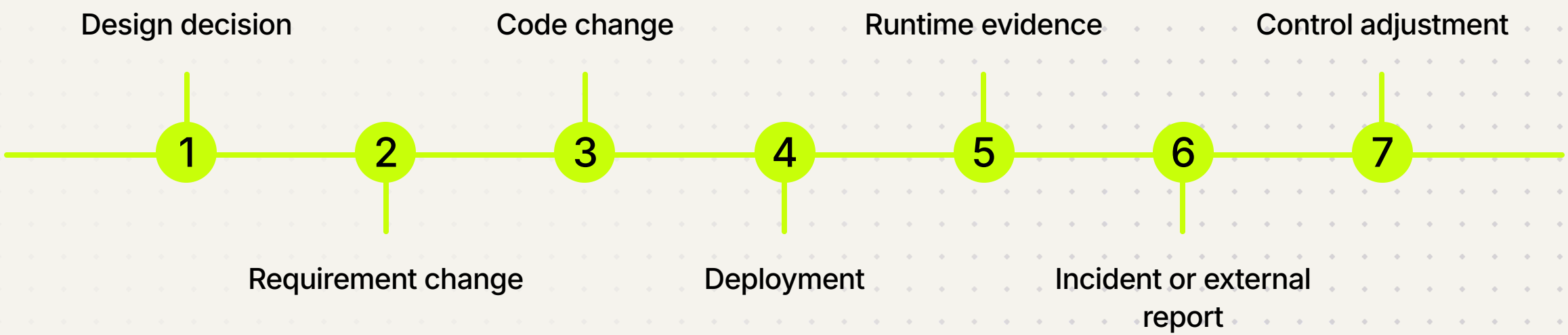
WEAKNESS

- Technology age and dependencies
- Known vulnerabilities and attack paths
- Threat-model coverage

CHANGE

- Control effectiveness
- Significant recent changes
- Incidents and external reports
- Accepted and accumulating risk

Risk state is dynamic



AppSec operates at two connected levels

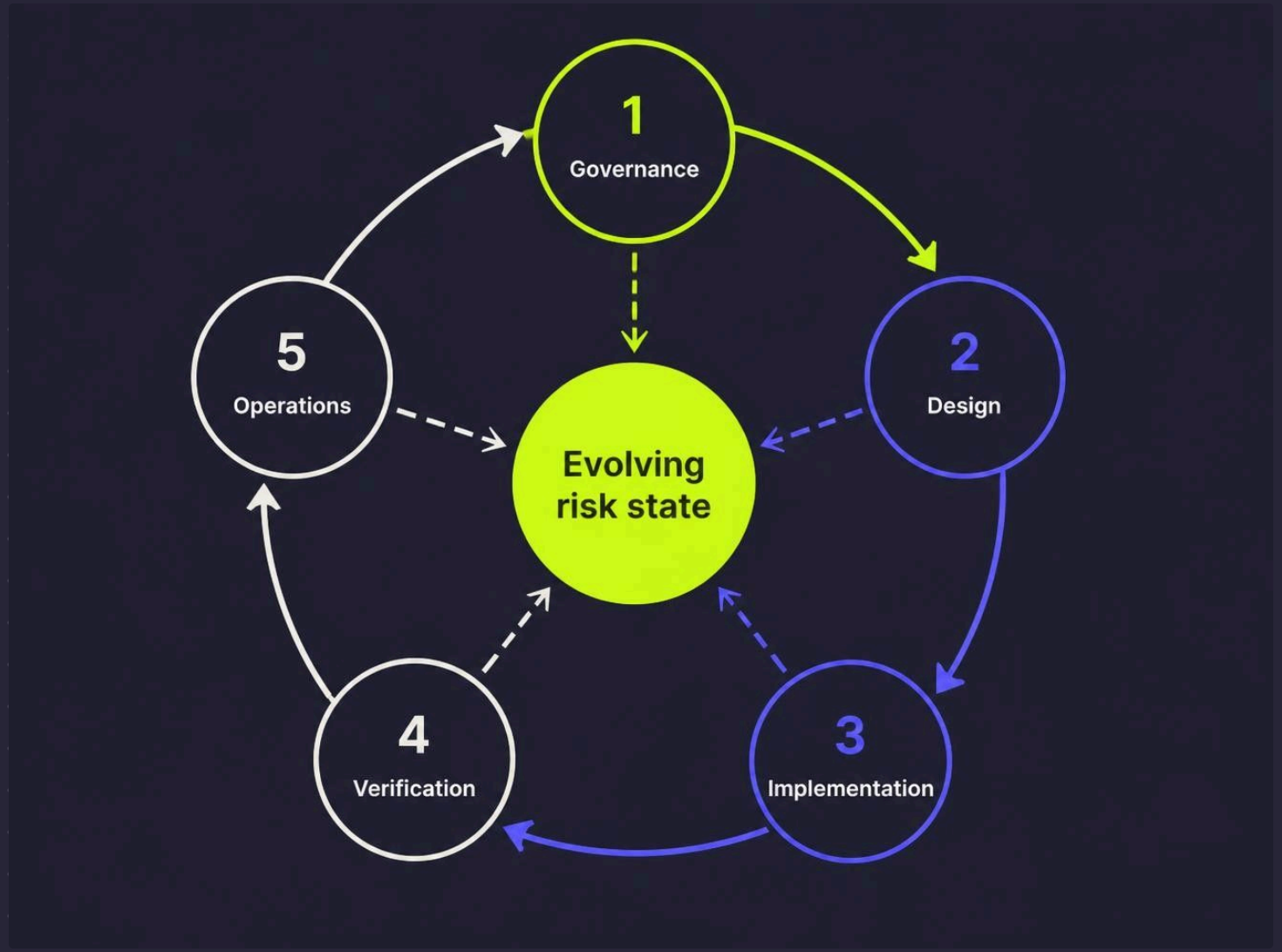
Service level

Maintain the evolving risk state. Evaluate significant changes. Keep threat and architecture context. Connect findings into attacker outcomes. Verify controls. Incorporate runtime evidence and incidents.

Program level

Define strategy and security properties. Build guardrails, secure frameworks, and paved roads. Govern agentic systems. Measure risk across services. Find categorical failures. Allocate expert attention.

AppSec already spans the entire software system



OWASP SAMM's five business functions as a connected loop, not pillars.

SAMM / GOVERNANCE

Know what matters, who owns it, and whether controls work

- Service-level risk
- Ownership
- Exposure and business importance
- Risk appetite

SECURITY POLICY

"Every customer-data lookup must be scoped to the authenticated user's tenant."

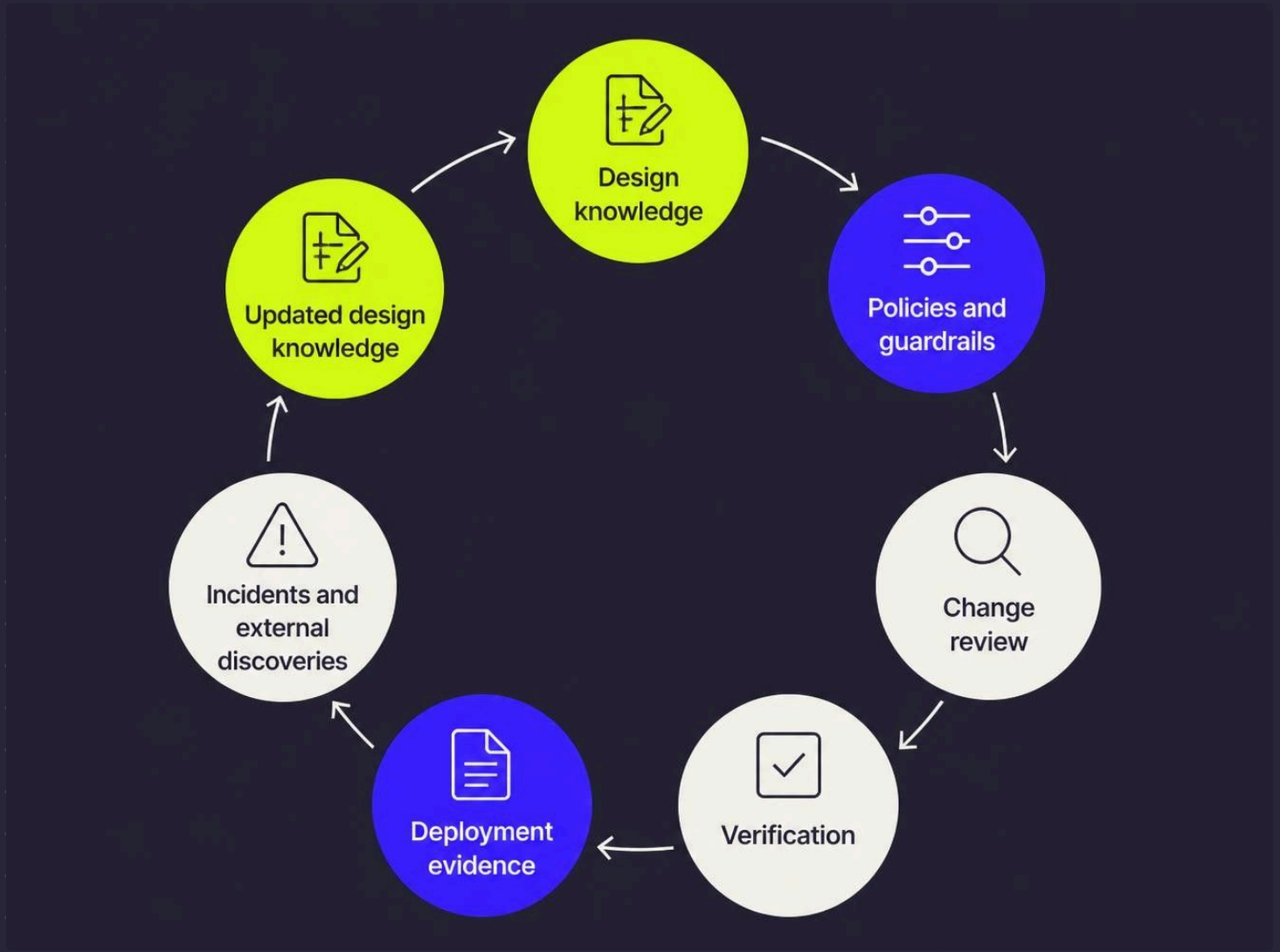
- Policies and security properties
- Control effectiveness
- Organizational trends

SECURITY PROPERTY

"A user can never read, modify, or delete an invoice belonging to another tenant, regardless of the identifier they supply."

A continuously updated decision layer.

Security knowledge follows the software



Every stage teaches the next.
Nothing is filed and forgotten.

The old product categories begin to dissolve

SAST

Isolated source findings → agentic investigation across changes, architecture, data flow, and context

DAST

Periodic external probing → hypothesis-driven attack exploration and runtime verification

SCA

Dependency inventory → reachability, exploitability, malicious package behavior, and application impact

Finding management

Ticket repository → evidence and application risk knowledge

Three observations can describe one attacker outcome

OBSERVATION - SAST

A client-controlled limit

OBSERVATION - SCA

A server that trusts the counter

OBSERVATION - ASSET INVENTORY

A paid upstream service

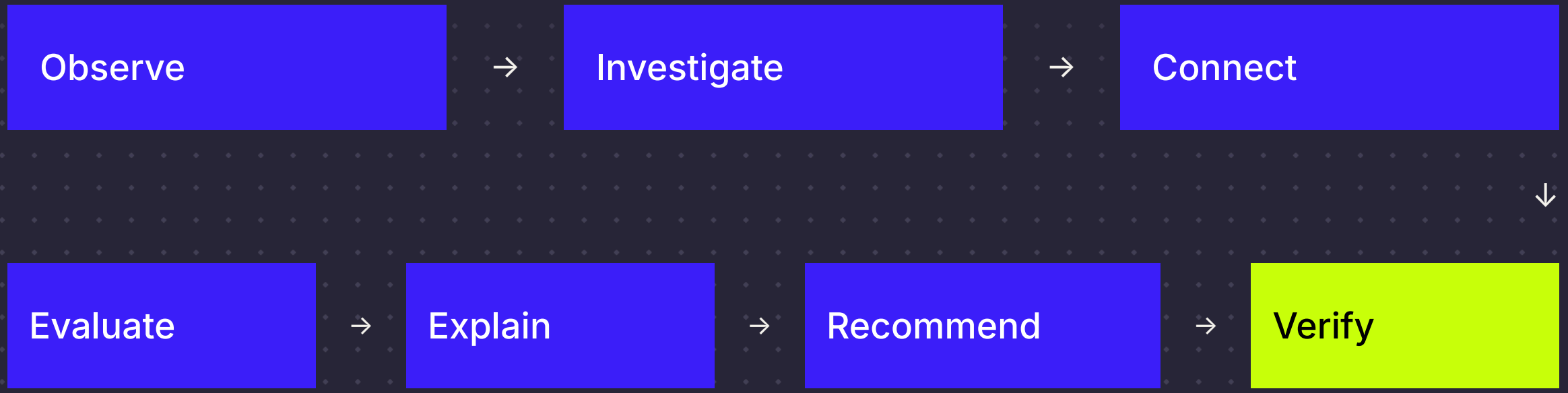


CONCLUSION

Unbounded paid API abuse

A generic example.

Intelligent systems should do more of the security work



The output is a reasoned conclusion with evidence, not a bigger inventory.

Human work moves upward

PRACTITIONERS ↑

Define security properties · challenge architecture · build guardrails · resolve ambiguity · make consequential risk decisions · govern the system

AGENTIC SYSTEMS

Gather context · inspect changes · maintain models · test hypotheses · correlate evidence · verify controls · track risk state

The hackers and the builders move closer together

APPSEC

Adversarial thinking, investigation, exploit reasoning, risk judgment

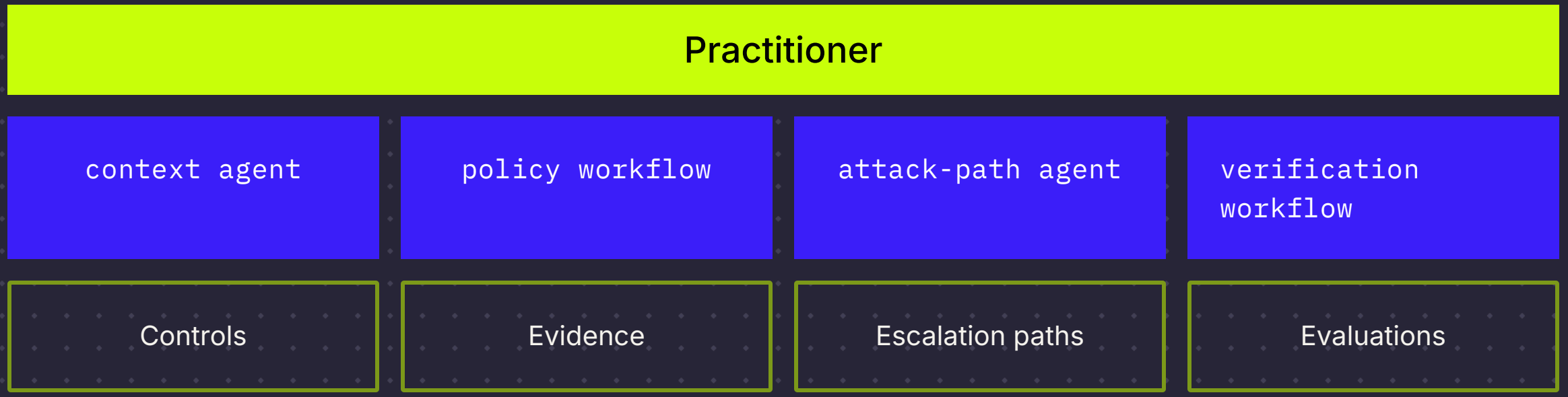
SHARED

Software-literate practitioners who can both challenge and build

PRODUCT SECURITY

Guardrails, secure libraries, platform integration, developer enablement

The future AppSec practitioner governs systems of work



ACT IV

How the program actually changes

Turning the vision into an operating model

Human authority moves to the boundaries of the system

NOVEL DISCOVERY

New vulnerability classes, attacker techniques, and architectural failure modes

AMBIGUITY

Incomplete evidence, conflicting signals, and business-dependent judgment



CONSEQUENTIAL DECISIONS

Risk acceptance, release blocking, architectural change, and evidence sufficiency

SYSTEM GOVERNANCE

Security properties, permissions, escalation paths, evaluations, and acceptable autonomy

Humans stop being the integration layer inside every transaction. They remain the authority at the system's most consequential boundaries.

TRUST IS AN ARCHITECTURAL PROPERTY

Trust the system, not the agent

Confidence should come from the design of the workflow, not from assuming the model is intelligent, careful, or correct.



Agent

■ BOUNDED SCOPE

The agent receives a specific task and a defined security property to evaluate.

■ LIMITED AUTHORITY

It can inspect code and propose a change, but it cannot merge or deploy it.

■ REQUIRED EVIDENCE

It must show the relevant code path, security control, tests, and reasoning behind its conclusion.

■ VERIFICATION + ESCALATION

Tests verify the security property. Ambiguous or consequential decisions go to a human owner.

Security property: A user must never access an invoice belonging to another tenant. The agent may trace the authorization path, identify a missing tenant check, write a test, and propose a patch. It may not merge or deploy that patch until the required evidence and approval conditions are satisfied.

The question is not, "Do we trust the agent?" The question is, "When the agent is wrong, what limits the consequences, exposes the error, and routes the decision appropriately?"

Autonomy expands with evidence

Each increase in authority is earned. None is assumed.

1

OBSERVE AND RECOMMEND

Gather evidence and propose an action

2

SUPPORT HUMAN DECISIONS

Produce a reasoned conclusion with evidence

3

TAKE BOUNDED, REVERSIBLE ACTIONS

Act within narrow permissions and defined rollback paths

4

ENFORCE EVALUATED CONTROLS

Apply controls that have demonstrated reliable behavior

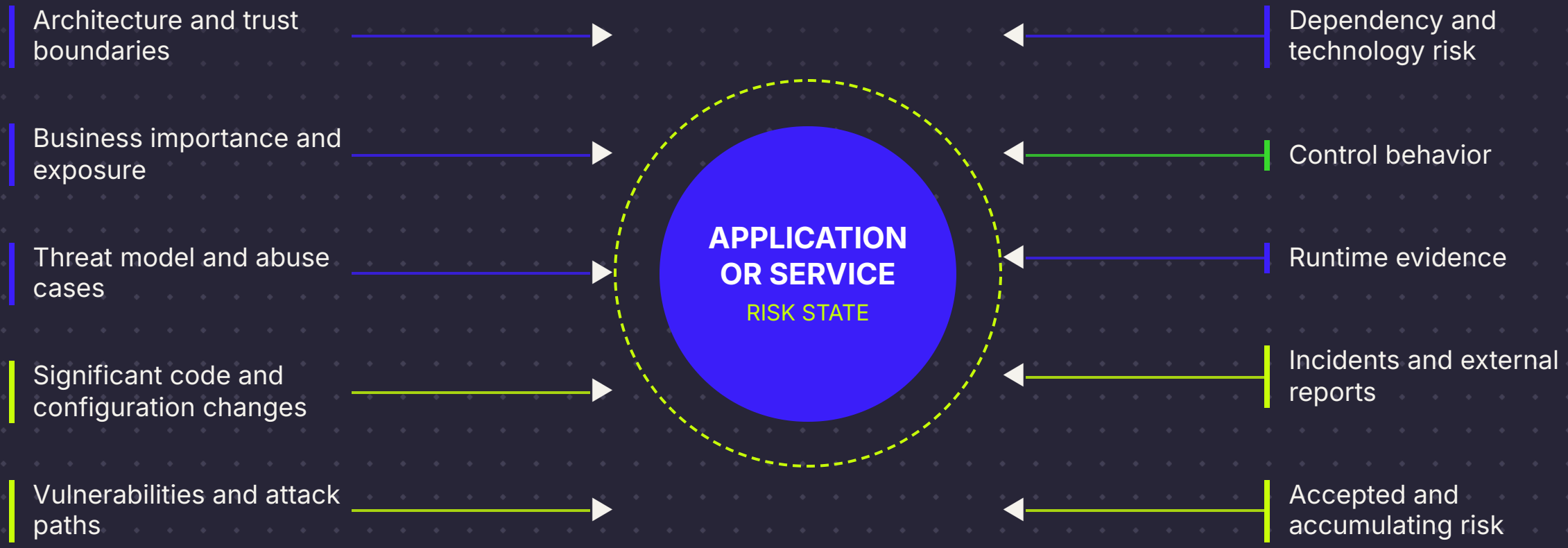
5

EXPAND AFTER DEMONSTRATED RELIABILITY

Grant additional authority based on evaluation and operational evidence

SERVICE LEVEL

Every important service develops a living risk state



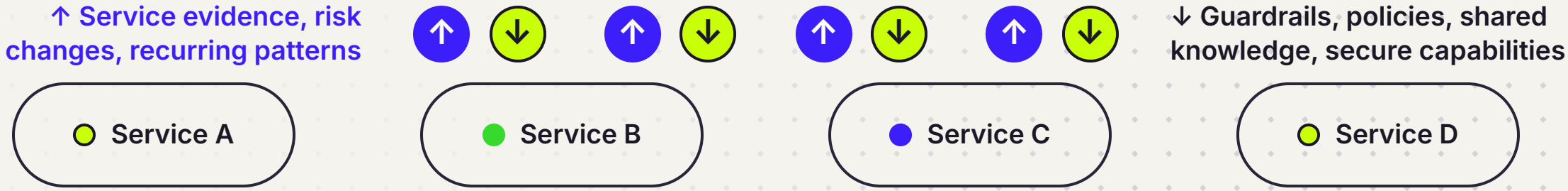
The service, not the individual finding, becomes the unit of security work.

PROGRAM LEVEL

The program sees and reduces systemic risk

PROGRAM-LEVEL APPSEC

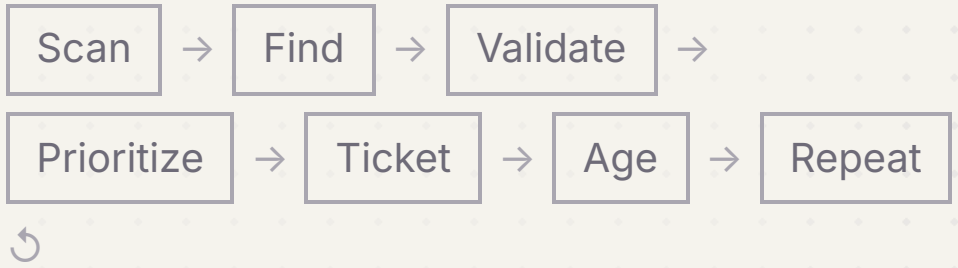
- Identify services accumulating meaningful risk
- Find recurring failure patterns across the organization
- Direct expert attention where it matters
- Improve classes of applications instead of processing findings individually
- Build reusable guardrails, policies, libraries, and paved roads
- Verify whether organizational controls are working
- Track ownership and accepted risk



Service evidence informs the program. Program capabilities improve every service.

The program no longer starts with the backlog

BACKLOG-CENTERED



The program organizes itself around processing tool output.

RISK-STATE-CENTERED

Understand the service

→ Observe meaningful change

→ Evaluate how risk changed

→ Verify controls

→ Update the risk state

→ Improve the system

The program organizes itself around reducing software risk.

Start with one important service

Do not begin by deploying a giant security agent across the company.

1

SELECT THE SERVICE

Real impact, active development.

2

ESTABLISH ITS RISK STATE

Architecture, ownership, exposure, controls.

3

CONNECT THE EVIDENCE

Design, code, runtime, incidents.

4

AUTOMATE THE CONTINUOUS WORK

Maintain context, verify controls.

5

REVIEW HOW RISK CHANGES

Humans decide; is risk improving?

Scale what works across the program



SERVICE PILOT

Prove the operating model on one important application.

REUSABLE KNOWLEDGE

Turn lessons, threat patterns, and decisions into reusable context.

SHARED CONTROLS

Policies, guardrails, secure libraries, safe defaults, verified workflows.

PORTFOLIO VISIBILITY

Compare service risk states; find organizational patterns.

EXPERT ALLOCATION

Direct experts to architecture, novel threats, and consequential decisions.

Scale the capability, not the manual process.

Measure changes in risk, not agent activity

DO NOT OPTIMIZE FOR

- Agent runs
- Findings generated
- Tickets opened
- Automated comments
- Apparent task completion
- Percentage of work labeled autonomous

OPTIMIZE FOR

SERVICE LEVEL

- Material attack paths removed
- Unsafe changes prevented
- Controls verified
- Exposure reduced
- Accepted risk made explicit
- Practitioner time returned

PROGRAM LEVEL

- Recurring risk patterns eliminated
- Shared guardrail adoption
- High-risk services improving
- Organizational controls producing evidence
- Expert attention reaching consequential work

AppSec gets its real job back

EXPERTISE CONSUMED BY THE PROCESS

- Validating scanner output
- Reconstructing missing context
- Moving tickets
- Chasing backlog age
- Repeating the same guidance
- Acting as middleware between tools and engineers

EXPERTISE APPLIED TO RISK

- Challenging architecture
- Discovering new failure modes
- Building secure capabilities
- Investigating meaningful attack paths
- Advising consequential decisions
- Reducing systemic software risk

AppSec becomes the control system for software risk, around the software organization rather than bolted onto the end.

The end of human-scale AppSec is not the end of the AppSec practitioner.

It is the end of using practitioners as middleware between security tools and engineering teams.

If the SDLC is being rebuilt, AppSec has to be rebuilt with it.

Ken Johnson
CTO & Co-Founder, DryRun Security
linkedin.com/in/cktricky
ken@dryrunsecurity.com